

ENGR 102 Summer Bridge

Day 16 Student Workbook

Indexed Mutation and Independent Exam 2 Rehearsal

Monday, July 27, 2026 • 50 points

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as **and**, **or**, and **not**. Their lowercase spelling is required in code.

Daily target

increasing independence

Design, mutate, summarize, and test a complete list-processing program from requirements alone.

Allowed tools	All Exam 2 tools: loops, conditions, list reads, append, len, and indexed assignment
Support supplied	Only the requirements, constraints, and test obligations are supplied.
You must decide	The entire algorithm, variable inventory, loop form, mutation order, summaries, and tests.
Scaffold level	Independent construction; no planning questions and no algorithm steps.

Scoring and completion standard

50 points

Evidence	Points	Secure work shows
Integrated trace	8	intermediate state, condition results, and exact output
Diagnosis and repair	6	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	36	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace**8 points**

Produce one complete mutation ledger and exact final output.

```
values = [-2, 7, 14, 3]
changed = 0

for index in range(len(values)):
    original = values[index]
    if original < 0:
        values[index] = 0
    elif original % 2 == 1:
        values[index] = original * 2
    else:
        continue
    changed = changed + 1

print(values, changed)
```

Your task

1. For every index, record original value, selected branch, final value, list state, and changed; then give exact output.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair**6 points**

The intent is to replace every negative value, including the last item, and count each replacement.

```
values = [4, -2, 8, -5]
changed = 0

for index in range(len(values) - 1):
    if values[index] < 0:
        values[index] = 0
        changed = changed + 1

print(values, changed)
```

Your task

1. Give actual and intended output, identify the boundary defect, repair it, and name a test that would fail if the last index were skipped again.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program**36 points across Pages 4-5****Program specification**

- Use temperatures = [18, -4, 31, 22, 50].
- Mutate every value below 0 to 0 and every value above 40 to 40.
- Count how many positions change.
- Using the final mutated values, compute total, average, and the count from 20 through 40 inclusive.
- Print the final list, changed count, total, average, and in_range count in that order on separate lines.
- Do not use sum, min, max, comprehensions, slicing, or helper functions. Provide two tests with expected results, including a no-change case and a first/last-change case.

Program workspace – begin the connected solution here.

Task 3 continued. Test and defend the program**included in 36 points**

Continue code if needed.

Required tests, expected results, and brief defense

--